In [ ]:
```
import pandas as pd

# Change names if needed to match left panel
glass_file = "glass.data"
diabetes_file = "diabetes.csv"

glass = pd.read_csv(glass_file, header=None)
diabetes = pd.read_csv(diabetes_file)

print("Glass dataset shape:", glass.shape)
print("Diabetes dataset shape:", diabetes.shape)

glass.head(), diabetes.head()
```

```
Glass dataset shape: (214, 11)
Diabetes dataset shape: (768, 9)
```

Out[ ]:
```
(   0        1      2     3     4      5     6     7    8    9   10
 0   1  1.52101  13.64  4.49  1.10  71.78  0.06  8.75  0.0  0.0   1
 1   2  1.51761  13.89  3.60  1.36  72.73  0.48  7.83  0.0  0.0   1
 2   3  1.51618  13.53  3.55  1.54  72.99  0.39  7.78  0.0  0.0   1
 3   4  1.51766  13.21  3.69  1.29  72.61  0.57  8.22  0.0  0.0   1
 4   5  1.51742  13.27  3.62  1.24  73.08  0.55  8.07  0.0  0.0   1,
    Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
 0            6      148             72             35        0  33.6
 1            1       85             66             29        0  26.6
 2            8      183             64              0        0  23.3
 3            1       89             66             23       94  28.1
 4            0      137             40             35      168  43.1

    DiabetesPedigreeFunction  Age  Outcome
 0                     0.627   50        1
 1                     0.351   31        0
 2                     0.672   32        1
 3                     0.167   21        0
 4                     2.288   33        1  )
```

In [ ]:
```
# R1 - preprocessing

import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler

# split features and labels
X_glass = glass.iloc[:, :-1]
y_glass = glass.iloc[:, -1]

X_diabetes = diabetes.iloc[:, :-1]
y_diabetes = diabetes.iloc[:, -1]

# replace medical zeros with missing values
cols_with_missing = [1, 2, 3, 4, 5]
X_diabetes.iloc[:, cols_with_missing] = X_diabetes.iloc[:, cols_with_missing].repla

# fill missing values
```

```python
imputer = SimpleImputer(strategy="median")
X_glass = imputer.fit_transform(X_glass)
X_diabetes = imputer.fit_transform(X_diabetes)

# scale features
scaler = MinMaxScaler()
X_glass = scaler.fit_transform(X_glass)
X_diabetes = scaler.fit_transform(X_diabetes)

print("R1 finished")
```

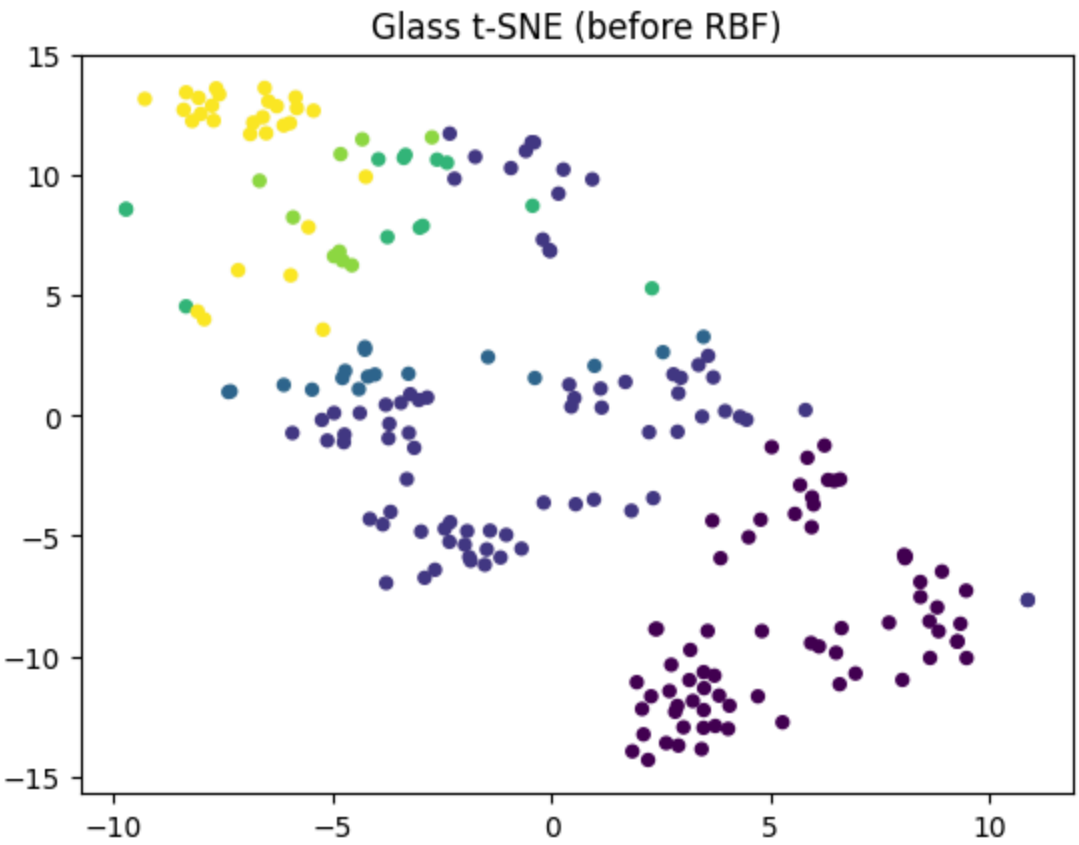
```
R1 finished
```

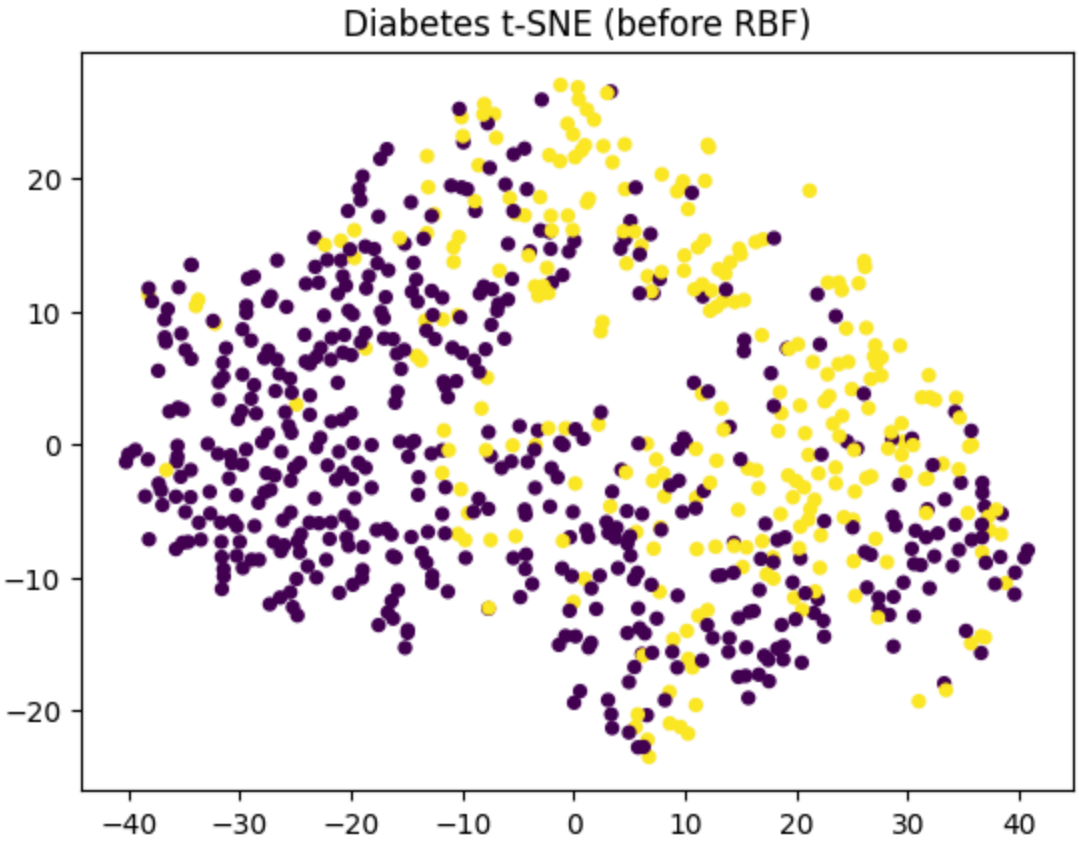
```
/tmp/ipython-input-1554747484.py:16: FutureWarning: Setting an item of incompatible
dtype is deprecated and will raise in a future error of pandas. Value '0      148.0
1       85.0
2      183.0
3       89.0
4      137.0
       ...
763    101.0
764    122.0
765    121.0
766    126.0
767     93.0
Name: Glucose, Length: 768, dtype: float64' has dtype incompatible with int64, pleas
e explicitly cast to a compatible dtype first.
  X_diabetes.iloc[:, cols_with_missing] = X_diabetes.iloc[:, cols_with_missing].repl
ace(0, np.nan)
/tmp/ipython-input-1554747484.py:16: FutureWarning: Setting an item of incompatible
dtype is deprecated and will raise in a future error of pandas. Value '0      72.0
1      66.0
2      64.0
3      66.0
4      40.0
       ...
763    76.0
764    70.0
765    72.0
766    60.0
767    70.0
Name: BloodPressure, Length: 768, dtype: float64' has dtype incompatible with int64,
please explicitly cast to a compatible dtype first.
  X_diabetes.iloc[:, cols_with_missing] = X_diabetes.iloc[:, cols_with_missing].repl
ace(0, np.nan)
/tmp/ipython-input-1554747484.py:16: FutureWarning: Setting an item of incompatible
dtype is deprecated and will raise in a future error of pandas. Value '0      35.0
1      29.0
2       NaN
3      23.0
4      35.0
       ...
763    48.0
764    27.0
765    23.0
766     NaN
767    31.0
Name: SkinThickness, Length: 768, dtype: float64' has dtype incompatible with int64,
please explicitly cast to a compatible dtype first.
  X_diabetes.iloc[:, cols_with_missing] = X_diabetes.iloc[:, cols_with_missing].repl
ace(0, np.nan)
/tmp/ipython-input-1554747484.py:16: FutureWarning: Setting an item of incompatible
dtype is deprecated and will raise in a future error of pandas. Value '0        NaN
1        NaN
2        NaN
3       94.0
4      168.0
       ...
763    180.0
```

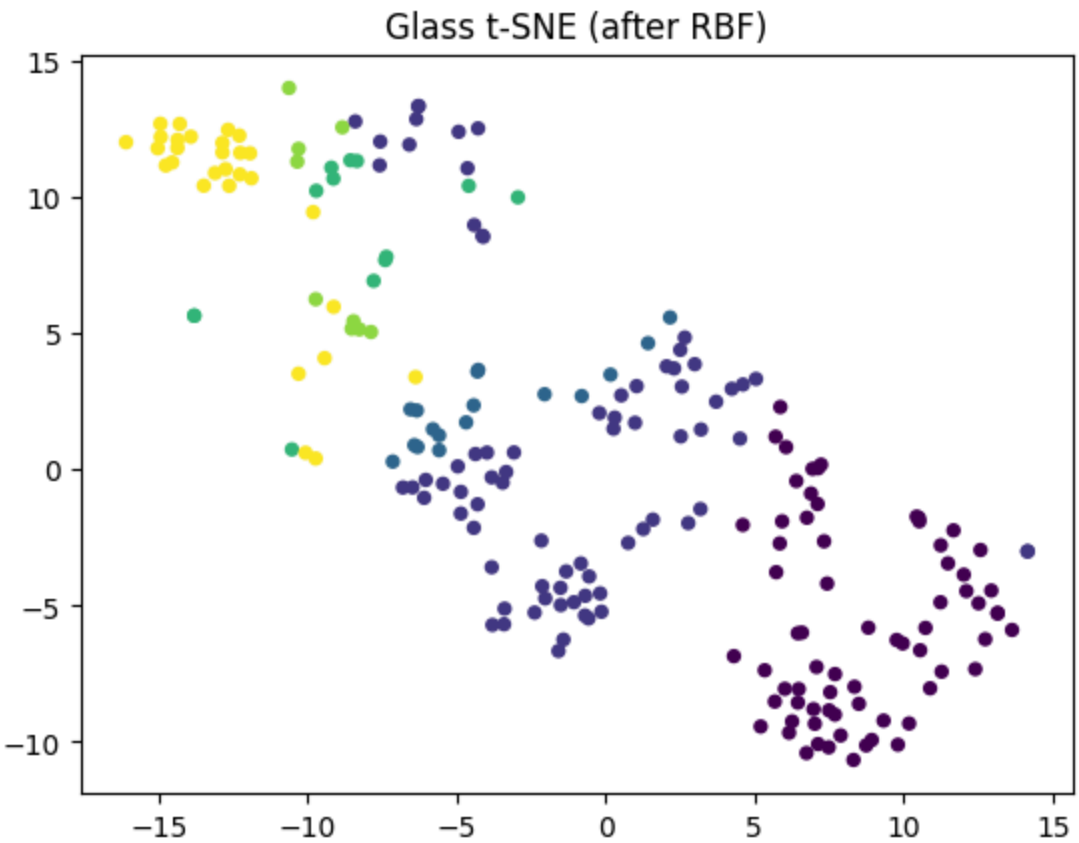
```
764      NaN
765    112.0
766      NaN
767      NaN
Name: Insulin, Length: 768, dtype: float64' has dtype incompatible with int64, pleas
e explicitly cast to a compatible dtype first.
  X_diabetes.iloc[:, cols_with_missing] = X_diabetes.iloc[:, cols_with_missing].repl
ace(0, np.nan)
```

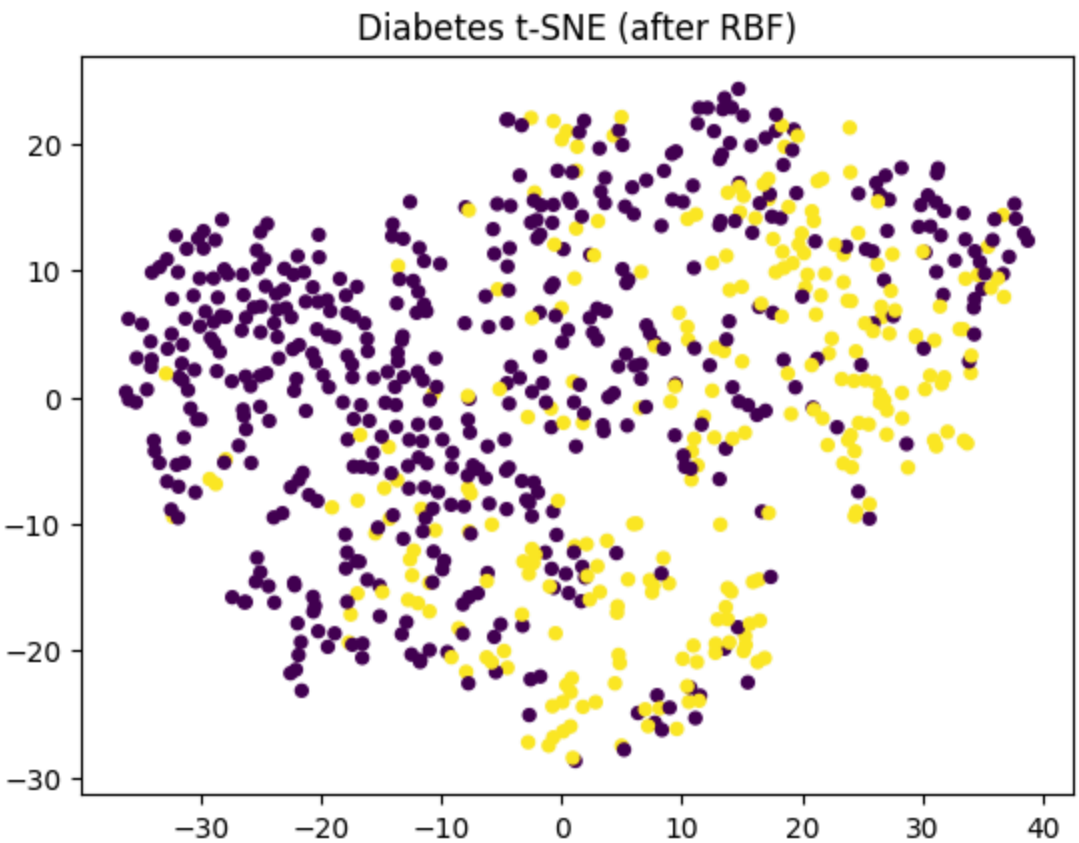
In [ ]:

```python
# R1(c,d) - RBF transform + t-SNE before/after

from sklearn.kernel_approximation import RBFSampler
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

def tsne_plot(X, y, title):
    Z = TSNE(n_components=2, random_state=42).fit_transform(X)
    plt.figure()
    plt.scatter(Z[:, 0], Z[:, 1], c=y, s=18)
    plt.title(title)
    plt.show()

# before kernel
tsne_plot(X_glass, y_glass, "Glass t-SNE (before RBF)")
tsne_plot(X_diabetes, y_diabetes, "Diabetes t-SNE (before RBF)")

# RBF kernel transform (approx)
rbf_glass = RBFSampler(gamma=1.0, n_components=500, random_state=42)
X_glass_rbf = rbf_glass.fit_transform(X_glass)

rbf_diab = RBFSampler(gamma=1.0, n_components=500, random_state=42)
X_diabetes_rbf = rbf_diab.fit_transform(X_diabetes)

# after kernel
tsne_plot(X_glass_rbf, y_glass, "Glass t-SNE (after RBF)")
tsne_plot(X_diabetes_rbf, y_diabetes, "Diabetes t-SNE (after RBF)")
```


Glass t-SNE (before RBF)
15
10
5
0
−5
−10
−15
−10
−5
0
5
10


Diabetes t-SNE (before RBF)
20
10
0
−10
−20
−40
−30
−20
−10
0
10
20
30
40





In [ ]:
```
# R2 - model training

from sklearn.model_selection import train_test_split
```

```python
from sklearn.linear_model import SGDClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import f1_score

# split data
Xg_train, Xg_test, yg_train, yg_test = train_test_split(
    X_glass, y_glass, test_size=0.3, random_state=42
)

Xd_train, Xd_test, yd_train, yd_test = train_test_split(
    X_diabetes, y_diabetes, test_size=0.3, random_state=42
)

# perceptron
perc = SGDClassifier(loss="perceptron")
perc.fit(Xg_train, yg_train)
f1_pg = f1_score(yg_test, perc.predict(Xg_test), average="weighted")

perc.fit(Xd_train, yd_train)
f1_pd = f1_score(yd_test, perc.predict(Xd_test), average="weighted")

# logistic regression
logreg = SGDClassifier(loss="log_loss")
logreg.fit(Xg_train, yg_train)
f1_lg = f1_score(yg_test, logreg.predict(Xg_test), average="weighted")

logreg.fit(Xd_train, yd_train)
f1_ld = f1_score(yd_test, logreg.predict(Xd_test), average="weighted")

# MLP
mlp = MLPClassifier(max_iter=500)
mlp.fit(Xg_train, yg_train)
f1_mg = f1_score(yg_test, mlp.predict(Xg_test), average="weighted")

mlp.fit(Xd_train, yd_train)
f1_md = f1_score(yd_test, mlp.predict(Xd_test), average="weighted")

print("Glass dataset scores:")
print("Perceptron:", f1_pg)
print("Logistic:", f1_lg)
print("MLP:", f1_mg)

print("\nDiabetes dataset scores:")
print("Perceptron:", f1_pd)
print("Logistic:", f1_ld)
print("MLP:", f1_md)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptro
n.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached
and the optimization hasn't converged yet.
  warnings.warn(
```

```
Glass dataset scores:
Perceptron: 0.8250144444781982
Logistic: 0.7316559589835453
MLP: 0.9193891693891694

Diabetes dataset scores:
Perceptron: 0.6343720558596592
Logistic: 0.6908158922965902
MLP: 0.7328810115788985
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptro
n.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached
and the optimization hasn't converged yet.
  warnings.warn(
```

In [ ]:

```python
# R2 - repeated 5-fold CV + tuning (max_iter, gamma) + MLP tuning

import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV, RepeatedStratif
from sklearn.pipeline import Pipeline
from sklearn.linear_model import SGDClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import f1_score, make_scorer
from sklearn.kernel_approximation import RBFSampler

f1w = make_scorer(f1_score, average="weighted")
cv5 = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)

def tune_max_iter(Xtr, ytr, loss_name):
    base = SGDClassifier(loss=loss_name, random_state=42)
    grid = {"max_iter": [200, 500, 1000, 2000, 5000]}
    gs = GridSearchCV(base, grid, scoring=f1w, cv=cv5, n_jobs=-1)
    gs.fit(Xtr, ytr)
    return gs.best_params_["max_iter"], gs.best_score_

def tune_gamma(Xtr, ytr, loss_name, best_iter):
    gammas = [0.001, 0.01, 0.1, 1.0, 5.0, 10.0]
    best_g, best_score = None, -1
    for g in gammas:
        model = Pipeline([
            ("rbf", RBFSampler(gamma=g, n_components=500, random_state=42)),
            ("clf", SGDClassifier(loss=loss_name, max_iter=best_iter, random_state=
        ])
        score = cross_val_score(model, Xtr, ytr, scoring=f1w, cv=cv5, n_jobs=-1).me
        if score > best_score:
            best_score = score
            best_g = g
    return best_g, best_score

def tune_mlp(Xtr, ytr):
    mlp = MLPClassifier(random_state=42)
    grid = {
        "max_iter": [300, 600, 1200],
        "n_iter_no_change": [10, 20, 30]
    }
    gs = GridSearchCV(mlp, grid, scoring=f1w, cv=cv5, n_jobs=-1)
    gs.fit(Xtr, ytr)
```

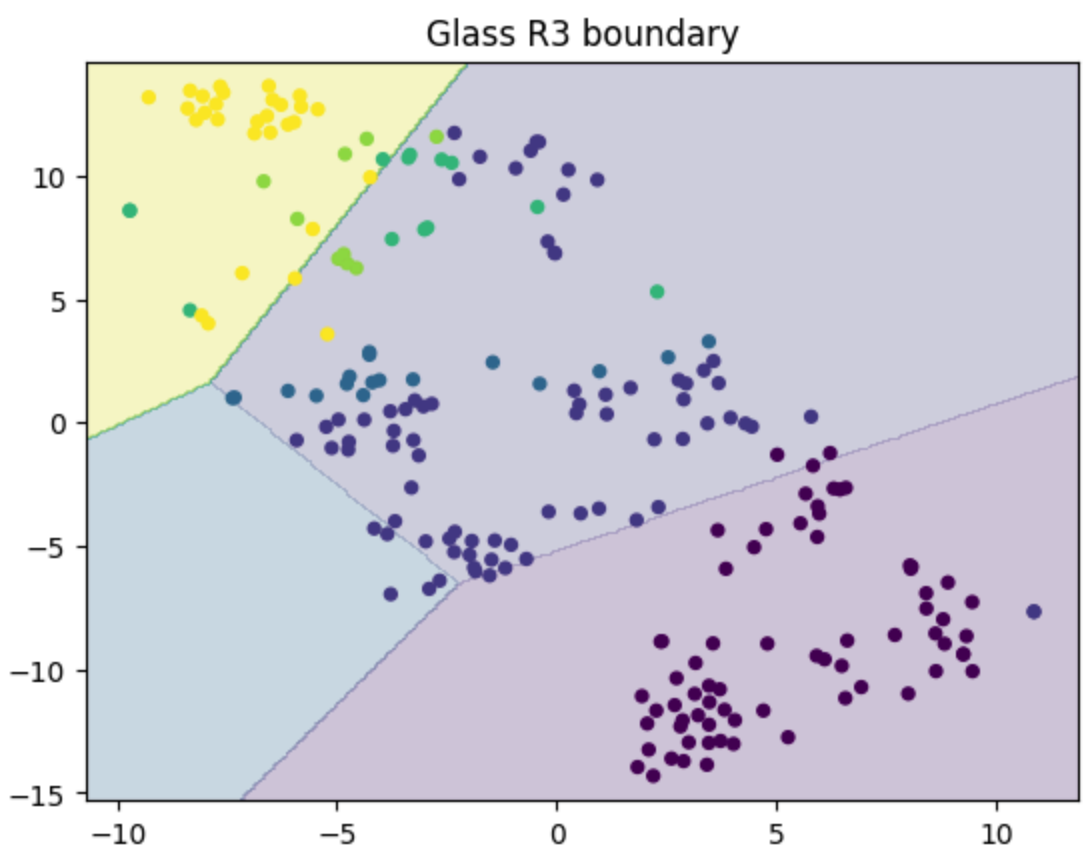
```
    return gs.best_params_, gs.best_score_

def run_r2_for_dataset(X, y, name):
    print("\n" + "="*60)
    print(name)
    print("="*60)

    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.30, random_state=42, st

    # Perceptron (no RBF)
    it_p, cv_p = tune_max_iter(Xtr, ytr, "perceptron")
    perc = SGDClassifier(loss="perceptron", max_iter=it_p, random_state=42)
    perc.fit(Xtr, ytr)
    f1_test_p = f1_score(yte, perc.predict(Xte), average="weighted")

    # Perceptron (with RBF)
    g_p, cv_p_rbf = tune_gamma(Xtr, ytr, "perceptron", it_p)
    perc_rbf = Pipeline([
        ("rbf", RBFSampler(gamma=g_p, n_components=500, random_state=42)),
        ("clf", SGDClassifier(loss="perceptron", max_iter=it_p, random_state=42))
    ])
    perc_rbf.fit(Xtr, ytr)
    f1_test_p_rbf = f1_score(yte, perc_rbf.predict(Xte), average="weighted")

    # Logistic Regression (no RBF)
    it_l, cv_l = tune_max_iter(Xtr, ytr, "log_loss")
    logreg = SGDClassifier(loss="log_loss", max_iter=it_l, random_state=42)
    logreg.fit(Xtr, ytr)
    f1_test_l = f1_score(yte, logreg.predict(Xte), average="weighted")

    # Logistic Regression (with RBF)
    g_l, cv_l_rbf = tune_gamma(Xtr, ytr, "log_loss", it_l)
    logreg_rbf = Pipeline([
        ("rbf", RBFSampler(gamma=g_l, n_components=500, random_state=42)),
        ("clf", SGDClassifier(loss="log_loss", max_iter=it_l, random_state=42))
    ])
    logreg_rbf.fit(Xtr, ytr)
    f1_test_l_rbf = f1_score(yte, logreg_rbf.predict(Xte), average="weighted")

    # MLP (tune max_iter + n_iter_no_change)
    mlp_params, cv_mlp = tune_mlp(Xtr, ytr)
    mlp = MLPClassifier(random_state=42, **mlp_params)
    mlp.fit(Xtr, ytr)
    f1_test_mlp = f1_score(yte, mlp.predict(Xte), average="weighted")

    print("CV Perceptron noRBF:", cv_p,      "| Test:", f1_test_p,      "| best_ite
    print("CV Perceptron RBF:  ", cv_p_rbf,  "| Test:", f1_test_p_rbf,  "| gamma:",
    print("CV LogReg noRBF:    ", cv_l,      "| Test:", f1_test_l,      "| best_ite
    print("CV LogReg RBF:      ", cv_l_rbf,  "| Test:", f1_test_l_rbf,  "| gamma:",
    print("CV MLP:             ", cv_mlp,    "| Test:", f1_test_mlp,    "| params:"

    lin_scores = {
        "perceptron": f1_test_p,
        "perceptron_rbf": f1_test_p_rbf,
        "logreg": f1_test_l,
        "logreg_rbf": f1_test_l_rbf
```

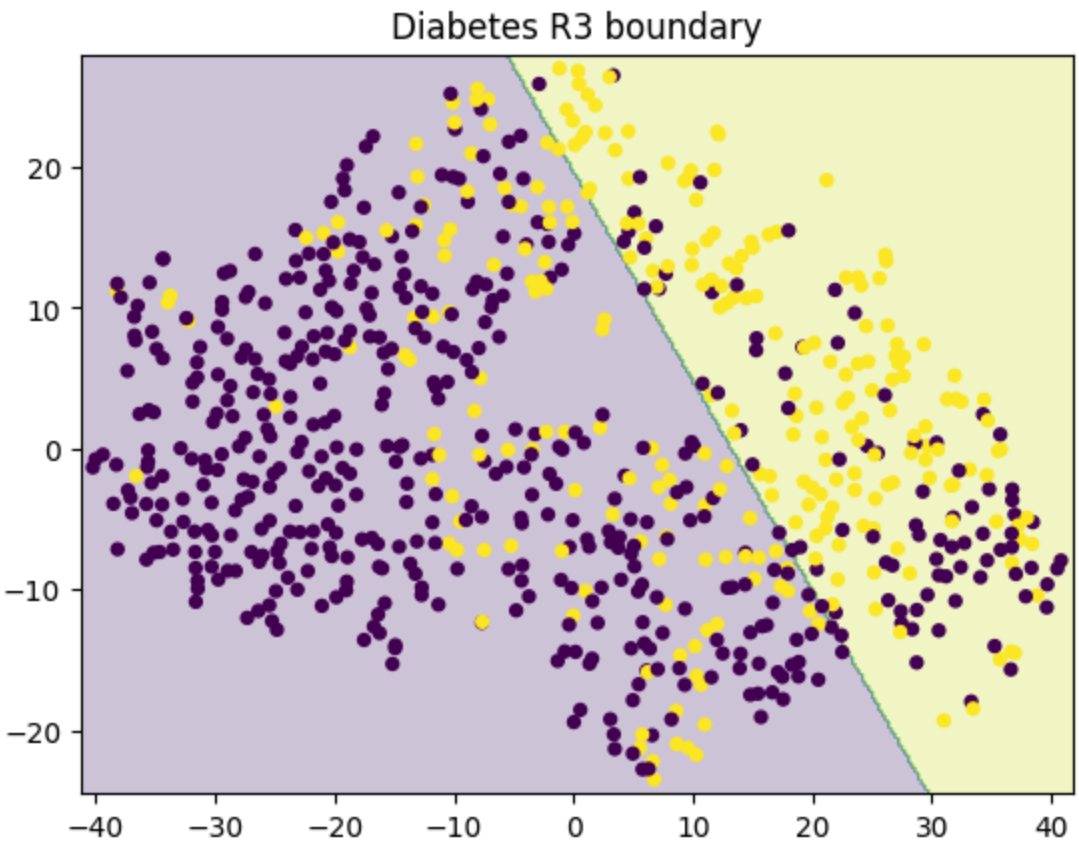
```
    }
    best_lin = max(lin_scores, key=lin_scores.get)
    print("Best linear (test):", best_lin, lin_scores[best_lin])

    return {
        "best_linear_name": best_lin,
        "best_iter_perc": it_p,
        "best_iter_logreg": it_l,
        "best_gamma_perc": g_p,
        "best_gamma_logreg": g_l
    }

# run required R2 for both datasets (these variables are what your check looks for)
r2_glass = run_r2_for_dataset(X_glass, y_glass, "Glass R2 (CV + tuning)")
r2_diab  = run_r2_for_dataset(X_diabetes, y_diabetes, "Diabetes R2 (CV + tuning)")
```

```
============================================================
Glass R2 (CV + tuning)
============================================================
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptro
n.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1200) reache
d and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptro
n.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1200) reache
d and the optimization hasn't converged yet.
  warnings.warn(
CV Perceptron noRBF: 0.6257643815817474 | Test: 0.8093447293447293 | best_iter: 200
CV Perceptron RBF:   0.8682394443781987 | Test: 0.9165018315018315 | gamma: 5.0
CV LogReg noRBF:     0.7131122550840305 | Test: 0.9036930124139424 | best_iter: 200
CV LogReg RBF:       0.8900070488041318 | Test: 0.948225371120108 | gamma: 5.0
CV MLP:              0.9261107932282159 | Test: 0.9701420530688822 | params: {'max_i
ter': 1200, 'n_iter_no_change': 10}
Best linear (test): logreg_rbf 0.948225371120108

============================================================
Diabetes R2 (CV + tuning)
============================================================
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptro
n.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached
and the optimization hasn't converged yet.
  warnings.warn(
CV Perceptron noRBF: 0.6416699129317724 | Test: 0.7242091726984337 | best_iter: 200
CV Perceptron RBF:   0.7026529925389605 | Test: 0.5112997238981491 | gamma: 0.001
CV LogReg noRBF:     0.7552837521962232 | Test: 0.6926117360899969 | best_iter: 200
CV LogReg RBF:       0.7553482818843643 | Test: 0.7358799558213048 | gamma: 1.0
CV MLP:              0.7736146875574678 | Test: 0.7437029637029636 | params: {'max_i
ter': 300, 'n_iter_no_change': 10}
Best linear (test): logreg_rbf 0.7358799558213048
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptro
n.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached
and the optimization hasn't converged yet.
  warnings.warn(
```

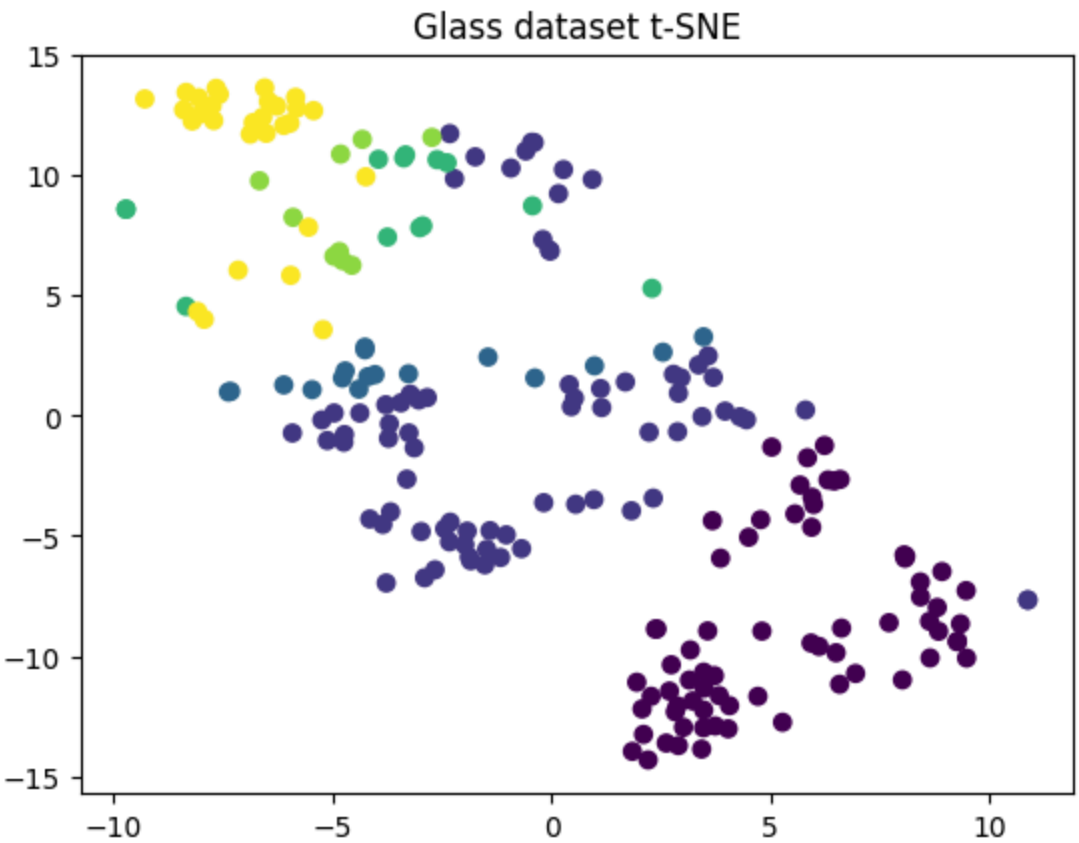
In [ ]:
```
# R3 boundary plot
```

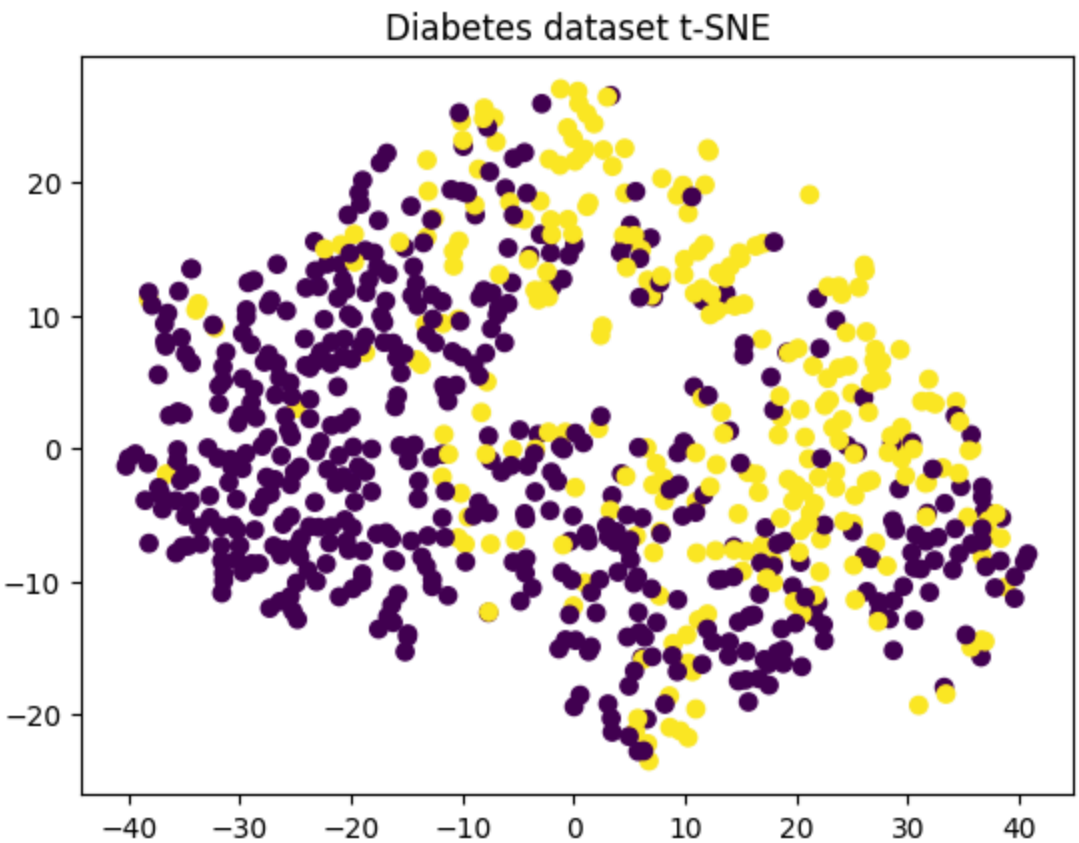
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.linear_model import SGDClassifier

def boundary_plot(X, y, title):
    Z = TSNE(n_components=2, random_state=42).fit_transform(X)

    clf = SGDClassifier(loss="log_loss", max_iter=2000, random_state=42)
    clf.fit(Z, y)

    x_min, x_max = Z[:,0].min()-1, Z[:,0].max()+1
    y_min, y_max = Z[:,1].min()-1, Z[:,1].max()+1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300)
    )

    grid = np.c_[xx.ravel(), yy.ravel()]
    pred = clf.predict(grid).reshape(xx.shape)

    plt.figure()
    plt.contourf(xx, yy, pred, alpha=0.25)
    plt.scatter(Z[:,0], Z[:,1], c=y, s=18)
    plt.title(title)
    plt.show()

boundary_plot(X_glass, y_glass, "Glass R3 boundary")
boundary_plot(X_diabetes, y_diabetes, "Diabetes R3 boundary")
```





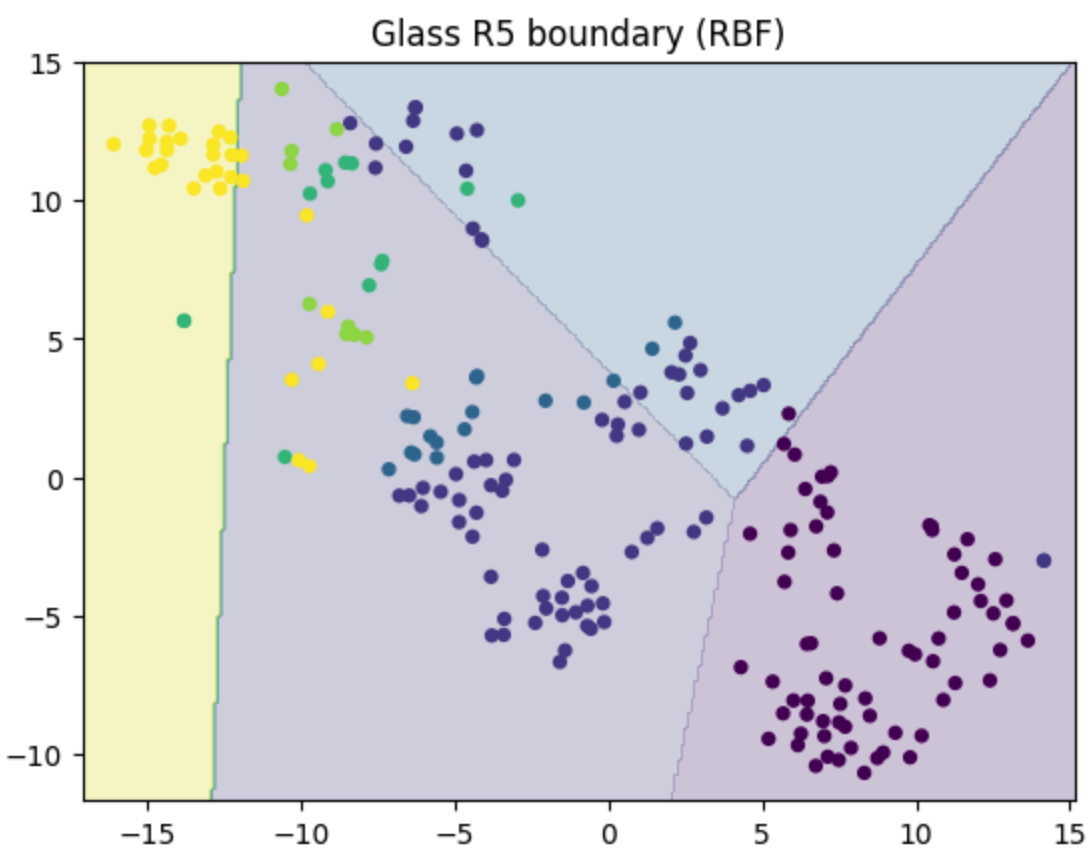
In [ ]:
```
# R3 - t-SNE visualization

from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# glass dataset
tsne_glass = TSNE(n_components=2, random_state=42)
glass_2d = tsne_glass.fit_transform(X_glass)

plt.figure()
plt.scatter(glass_2d[:, 0], glass_2d[:, 1], c=y_glass)
plt.title("Glass dataset t-SNE")
plt.show()

# diabetes dataset
tsne_diabetes = TSNE(n_components=2, random_state=42)
diabetes_2d = tsne_diabetes.fit_transform(X_diabetes)

plt.figure()
plt.scatter(diabetes_2d[:, 0], diabetes_2d[:, 1], c=y_diabetes)
plt.title("Diabetes dataset t-SNE")
plt.show()
```

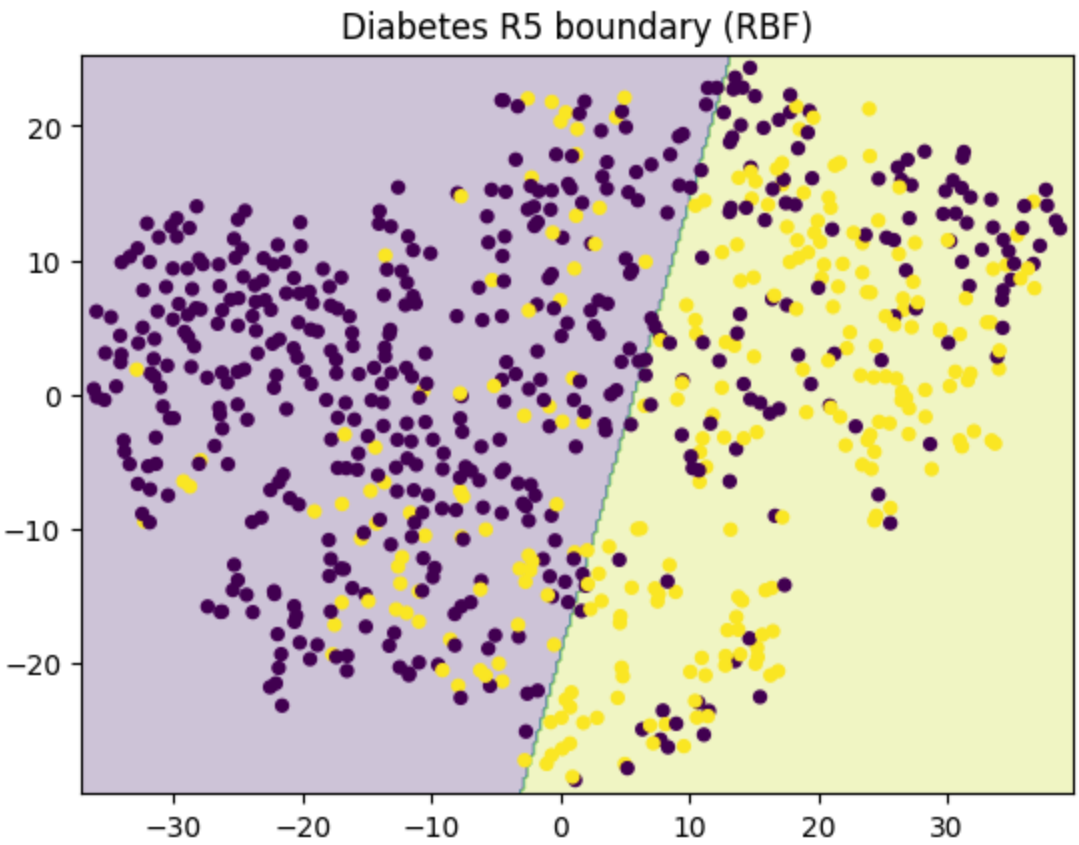




In [ ]:
```
# R5 boundary after RBF

from sklearn.kernel_approximation import RBFSampler
```

```
def boundary_plot_rbf(X, y, title):
    Xr = RBFSampler(gamma=1.0, n_components=500, random_state=42).fit_transform(X)

    Z = TSNE(n_components=2, random_state=42).fit_transform(Xr)

    clf = SGDClassifier(loss="log_loss", max_iter=2000, random_state=42)
    clf.fit(Z, y)

    x_min, x_max = Z[:,0].min()-1, Z[:,0].max()+1
    y_min, y_max = Z[:,1].min()-1, Z[:,1].max()+1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300)
    )

    grid = np.c_[xx.ravel(), yy.ravel()]
    pred = clf.predict(grid).reshape(xx.shape)

    plt.figure()
    plt.contourf(xx, yy, pred, alpha=0.25)
    plt.scatter(Z[:,0], Z[:,1], c=y, s=18)
    plt.title(title)
    plt.show()

boundary_plot_rbf(X_glass, y_glass, "Glass R5 boundary (RBF)")
boundary_plot_rbf(X_diabetes, y_diabetes, "Diabetes R5 boundary (RBF)")
```





In [ ]:
```
# R4 - feature importance (logistic regression) + drop 5 least important

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import f1_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler

def prep_diabetes_df(df):
    X = df.drop(columns=["Outcome"]).copy()
    y = df["Outcome"].copy()
    cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
    for c in cols:
        if c in X.columns:
            X.loc[X[c] == 0, c] = np.nan
    return X, y

def prep_glass_df(df):
    X = df.iloc[:, :-1].copy()
    y = df.iloc[:, -1].copy()
    X.columns = [f"F{i}" for i in range(X.shape[1])]  # glass has no header
    return X, y

def preprocess_to_scaled(X_df):
    imputer = SimpleImputer(strategy="median")
    scaler = MinMaxScaler()
    X_imp = imputer.fit_transform(X_df)
```

```
    X_scaled = scaler.fit_transform(X_imp)
    return X_scaled

def coef_importance(model, feature_names):
    coefs = model.coef_
    if coefs.ndim == 1:
        imp = np.abs(coefs)
    else:
        imp = np.mean(np.abs(coefs), axis=0)
    return pd.Series(imp, index=feature_names).sort_values(ascending=False)

def r4_run(X_df, y, name):
    X_scaled = preprocess_to_scaled(X_df)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.3, random_state=42, stratify=y
    )

    lr = SGDClassifier(loss="log_loss", class_weight=None, max_iter=2000, random_st
    lr.fit(X_train, y_train)

    imp = coef_importance(lr, X_df.columns)
    top5 = imp.head(5)
    bottom5 = imp.tail(5)

    pred = lr.predict(X_test)
    f1_before = f1_score(y_test, pred, average="weighted")

    X_df_drop = X_df.drop(columns=list(bottom5.index))
    X_scaled_drop = preprocess_to_scaled(X_df_drop)

    X_train2, X_test2, y_train2, y_test2 = train_test_split(
        X_scaled_drop, y, test_size=0.3, random_state=42, stratify=y
    )

    lr2 = SGDClassifier(loss="log_loss", class_weight=None, max_iter=2000, random_s
    lr2.fit(X_train2, y_train2)

    pred2 = lr2.predict(X_test2)
    f1_after = f1_score(y_test2, pred2, average="weighted")

    print("\n" + "="*60)
    print(name)
    print("="*60)
    print("Top 5 features:")
    print(top5)
    print("\nBottom 5 features:")
    print(bottom5)
    print("\nF1 (all features):", f1_before)
    print("F1 (dropped bottom 5):", f1_after)
    print("Dropped:", list(bottom5.index))

# run R4 for both datasets
Xg_df, yg_df = prep_glass_df(glass)
Xd_df, yd_df = prep_diabetes_df(diabetes)
```

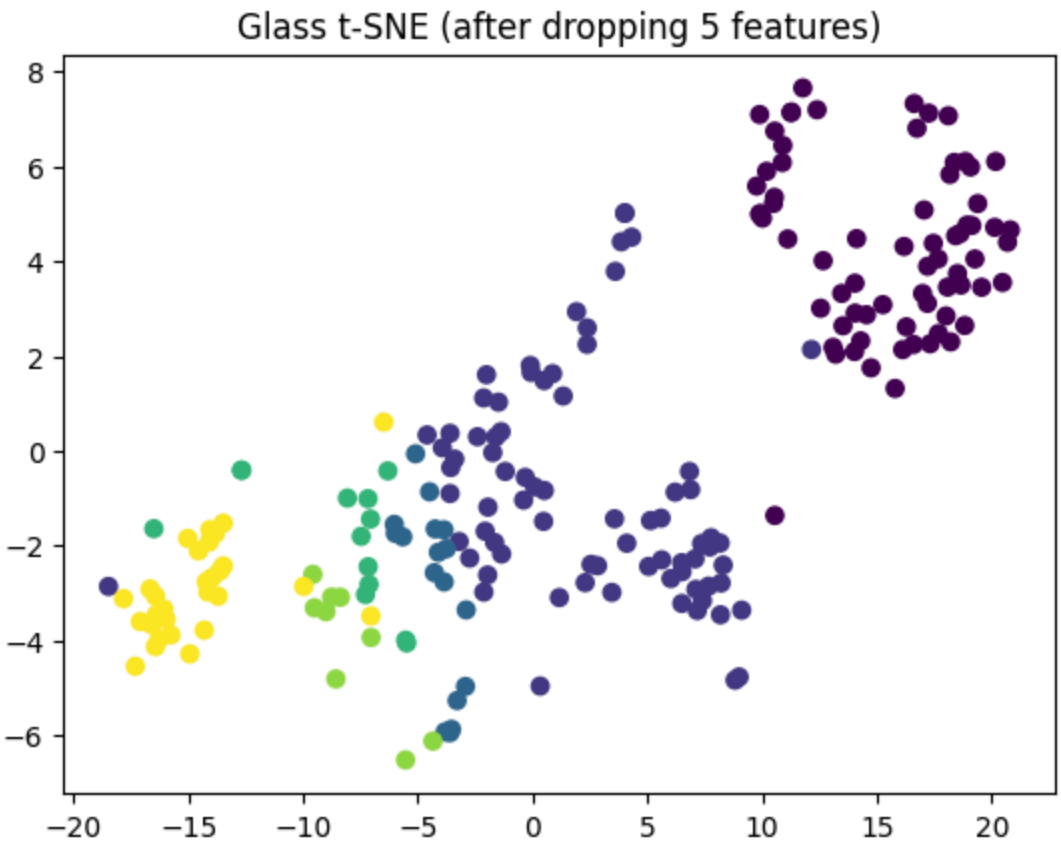
```
r4_run(Xg_df, yg_df, "Glass (R4)")
r4_run(Xd_df, yd_df, "Diabetes (R4)")
```

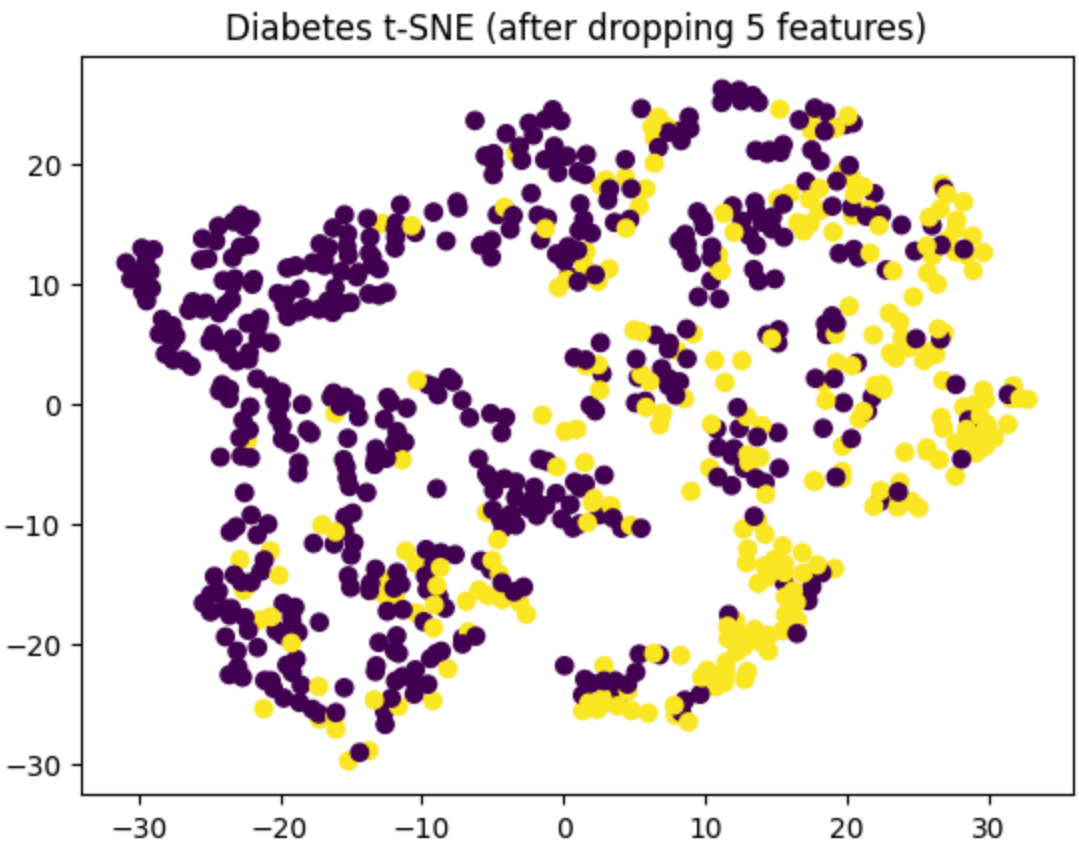
```
============================================================
Glass (R4)
============================================================
Top 5 features:
F0    19.294343
F2     7.992777
F4     7.964984
F8     7.316819
F1     5.605058
dtype: float64

Bottom 5 features:
F3    5.337596
F7    4.272539
F5    3.687677
F6    3.411700
F9    2.959081
dtype: float64

F1 (all features): 0.9036930124139424
F1 (dropped bottom 5): 0.8024424850511807
Dropped: ['F3', 'F7', 'F5', 'F6', 'F9']

============================================================
Diabetes (R4)
============================================================
Top 5 features:
Glucose                     6.854282
BMI                         5.793869
Pregnancies                 2.689183
DiabetesPedigreeFunction    2.158739
SkinThickness               1.240798
dtype: float64

Bottom 5 features:
DiabetesPedigreeFunction    2.158739
SkinThickness               1.240798
BloodPressure               0.479410
Age                         0.442182
Insulin                     0.116195
dtype: float64

F1 (all features): 0.6926117360899969
F1 (dropped bottom 5): 0.7208093111707569
Dropped: ['DiabetesPedigreeFunction', 'SkinThickness', 'BloodPressure', 'Age', 'Insu
lin']
```

In [ ]:

```
# R5 - t-SNE after dropping 5 least important features

from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
```

```python
def scale_df(X_df):
    imputer = SimpleImputer(strategy="median")
    scaler = MinMaxScaler()
    X_imp = imputer.fit_transform(X_df)
    X_scaled = scaler.fit_transform(X_imp)
    return X_scaled

# drop lists from your R4 output
drop_glass = ['F3', 'F7', 'F5', 'F6', 'F9']
drop_diab  = ['DiabetesPedigreeFunction', 'SkinThickness', 'BloodPressure', 'Age',

# ---- Glass ----
Xg_drop = Xg_df.drop(columns=drop_glass)
Xg_drop_scaled = scale_df(Xg_drop)

tsne_g = TSNE(n_components=2, random_state=42)
glass_2d_drop = tsne_g.fit_transform(Xg_drop_scaled)

plt.figure()
plt.scatter(glass_2d_drop[:, 0], glass_2d_drop[:, 1], c=yg_df)
plt.title("Glass t-SNE (after dropping 5 features)")
plt.show()

# ---- Diabetes ----
Xd_drop = Xd_df.drop(columns=drop_diab)
Xd_drop_scaled = scale_df(Xd_drop)

tsne_d = TSNE(n_components=2, random_state=42)
diab_2d_drop = tsne_d.fit_transform(Xd_drop_scaled)

plt.figure()
plt.scatter(diab_2d_drop[:, 0], diab_2d_drop[:, 1], c=yd_df)
plt.title("Diabetes t-SNE (after dropping 5 features)")
plt.show()
```





link of the colab

https://colab.research.google.com/drive/1Jd06prx8K_5y7bfU5Bb9CdlcPKvN1dSZ?usp=sharing